

DSA STUDY BLUEPRINT SERIES

97. Construct BST from Preorder

Reconstructing BSTs from Preorder lists

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Rebuild:** Build tree nodes using bounding check limits.
- Preorder value is root; recurse for left with bound update.

Memory & Execution Blueprint

Preorder value: 5 • Left child bound limit set to 5 •
Right child bound limit set to INT_MAX

C++ Implementation Reference

Standard code layout and syntax

```
int preIdx = 0;
TreeNode* bldBST(vector& pre, int limit) {
    if (preIdx == pre.size() || pre[preIdx] > limit) return nullptr;
    TreeNode* root = new TreeNode(pre[preIdx++]);
    root->left = bldBST(pre, root->val);
    root->right = bldBST(pre, limit);
    return root;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(H)$

Key Practices & Gotchas

- **Important:** This approach is faster than sorting preorder to get inorder lists.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	$[-\infty, +\infty]$	True $(-\infty < 15 < +\infty)$	Recurse Left child with range $[-\infty, 15]$
10 (Left)	$[-\infty, 15]$	True $(-\infty < 10 < 15)$	Recurse Right child with range $[10, 15]$

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.